

Project 1 -- A Watch Loop with OpenCode

Difficulty: easy · Uses: Concept 4 (in-session loop).

Build. Start a long task in your repo (for example, a script that sleeps for a while and then writes a file). Set up an in-session loop that checks every minute whether the task has finished, and tells you the moment it has.

Done when the loop notices the task finished, says so once, and you can stop it cleanly, and you never sat watching the terminal.

Start here ...

Today, we are not just going to learn Loop Engineering in theory, but we will build it practically.

This is our Project 1:

"A Watch Loop."

The simple goal of this project is:

We will start a long-running task, and OpenCode will automatically monitor that task.

It will check every few seconds.

And as soon as the task is complete, OpenCode will inform us.

So, we won't have to sit in front of the terminal and check repeatedly.

Let's start step-by-step.

STEP 1 — PROJECT FOLDER

First, I am creating a throwaway project.

A throwaway repo,
because the AI agent will automatically work with files.

Therefore, do not experiment on your important project at the start.

Open Windows Terminal / PowerShell.

Command:

```
mkdir watch-loop-demo
```

Now go inside the folder:

```
cd watch-loop-demo
```

And let's initialize a Git repository:

```
git init
```

Now our practice repository is ready.

STEP 2 — CREATING A LONG TASK

Now we need a task
that does not complete instantly.

For this, I will create a simple Python script.

File name:

```
long-task.py
```

In this, we will do a simple task.

The script will wait for some time,
then create a file.

Code:

```
import time

print("Long task started...")

time.sleep(60)

with open("task-complete.txt", "w") as f:
    f.write("Task completed successfully!")

print("Long task finished!")
```

The important thing here is:

```
time.sleep(60)
```

This means the script will wait for 60 seconds.

After that:

```
task-complete.txt
```

file will be created.

This will be our signal that the task is complete.

STEP 3 — STARTING THE TASK

Now let's start this task.

Command:

```
python long-task.py
```

In the terminal:

```
Long task started...
```

will show.

Now the script is waiting for 60 seconds.

But we will not wait here manually.
This is the practical part of our Watch Loop.

STEP 4 — STARTING OPENCODE

Now open a second terminal.

Go into the same project folder:

```
cd watch-loop-demo
```

And start OpenCode:

```
opencode
```

OpenCode will open in the terminal.

If you are using OpenCode for the first time,
you will first have to connect a provider.

In OpenCode:

```
/connect
```

run.

Then connect your available model/provider.

Once the connection is established,
we can use OpenCode with the project.

STEP 5 — INITIALIZING THE PROJECT

Now inside OpenCode:

```
/init
```

run.

Its purpose is for OpenCode to analyze the project and create AGENTS.md for project instructions.

Now our AI coding agent is ready to understand the project.

STEP 6 — WATCH LOOP PROMPT

Now I will give OpenCode a simple instruction.

I am not telling it
how to do everything manually.

I am only telling it the desired behavior.

Prompt:

"

Create an in-session watch loop for the running task.

Check every 10 seconds whether task-complete.txt exists.

If it does **not** exist, **check** again after 10 seconds.

As soon as the file appears, tell me that the task is complete.

Report completion only once **and** then stop.

Do **not** modify the **long-running** task."

Now understand an important thing here.

We have given the AI a repeatable workflow:

```
CHECK
↓
CONDITION
↓
WAIT
↓
CHECK AGAIN
```

And when the condition is true:

COMPLETE

↓

REPORT

↓

STOP

STEP 7 — OBSERVING THE LOOP

Now OpenCode will monitor the task.

First check:

does task-complete.txt exist?

If no:

Wait.

Then check again.

Then:

Wait.

Then check.

This will keep repeating.

The long task is not complete yet,
so the loop will continue.

STEP 8 — TASK COMPLETE

Now after 60 seconds are complete,
our Python script:

task-complete.txt

will be created.

Now when the Watch Loop checks next,

the condition will be TRUE.

The file is found.

Now the agent knows:

The task is complete.

And there is no need for the loop to run continuously here.

It will report the completion:

"Task completed."

And the loop will stop.

This is our Watch Loop.

UNDERSTAND THE CONCEPT

Now for a moment, let's connect this practical to the concepts of Loop Engineering.

We have:

```
LONG TASK
↓
WATCH LOOP
↓
CHECK
↓
STOPPING CONDITION
```

The heartbeat starts the loop here.

Work means:
checking the completion status of the task.

Condition:
whether task-complete.txt exists or not.

The state in this simple example is minimal,
but in real systems, state is important for saving

previous progress.

And stopping condition:

```
"File found."
```

After this condition, the loop stops.

SAFETY / LIMIT

Another important point.

An autonomous loop should not run indefinitely.

If the task never completes,
the loop will keep checking continuously.

Therefore, in real projects:

MAX TIME
MAX TRIES
or
MAX SPEND

limits should be defined.

We will keep this concept in mind
for our demo as well.

For example:

Maximum 2 minutes.

If the task does not complete in 2 minutes,
the loop should stop.

This is an important part of production-level loop design.

FINAL RECAP

So what did we build today?

A simple:

WATCH LOOP.

A long task was started.

OpenCode monitored it.

The status was checked every 10 seconds.

The loop kept running until the task was complete.

The file appeared.

Completion was detected.

The agent informed us.

And the loop stopped.

In simple words:

The job of a Watch Loop is:

"Don't wait — monitor."

And this is the main idea of Loop Engineering too:

To make AI not just a system that gives an answer once,
but a repeatable and controlled workflow.

Today's practical completes here.

In the next project, we will build the loop
with more useful real-world behavior.

Allah Hafiz (Goodbye)!